Optimize linalg::conjugated for noncomplex value types

Document #:	P3050
Date:	2023/11/15
Project:	Programming Language C++ LEWG

Reply-to:	Mark Hoemmen <mhoemmen@nvidia.com>

Contents

§ 1 Authors

- Mark Hoemmen (mhoemmen@nvidia.com) (NVIDIA)

§ 2 Revision history

- Revision 0 to be submitted for the post-Kona mailing 2023/11/15

§ 3 Abstract

We propose the following change to the C++ Working Paper. If an `mdspan` object `x` has noncomplex `value_type`, and if that `mdspan` does not already have accessor type `conjugated_accessor<A>` for some nested accessor type `A`, then we propose to change `conjugated(x)` just to return `x`.

4 Design justification

4.1 Introduction

LWG finished its review of P1673 at the Kona 2023 WG21 meeting. One reviewer (see Acknowledgments) pointed out that `linalg::conjugated` could be optimized by having it be the identity function if *`conj-if-needed`* would have been the identity function anyway on the input `mdspan`'s `value_type`. This paper proposes that change. Specifically, if an `mdspan` object `x` has noncomplex `value_type`, and if that `mdspan` does not already have accessor type `conjugated_accessor<A>` for some nested accessor type `A`, then we propose to change `conjugated(x)` just to return `x`.

This change has two observable effects.

1. The result's accessor type will be different. Instead of being `conjugated_accessor<A>` for some `A`, it will just be `A`.

2. If `x` has noncomplex `value_type`, then `conjugated(x)` will no longer have const `element_type`.

We consider Effect (2) acceptable for two reasons.

a. *`in-vector`*, *`in-matrix`*, and *`in-object`* already do not need to have const `element_type`. Users can pass in views-of-nonconst `mdspan` as read-only vector or matrix parameters. Thus, making the `element_type` of `conjugated(x)` nonconst would not break existing calls to `linalg` functions that take input vector or matrix parameters.

b. `conjugated(conjugated(z))` for `z` with nonconst complex `element_type` already has nonconst `element_type`. Thus, generic code that depends on the `element_type` of the result of `conjugated` already cannot assume that it is const.

4.2 Current behavior of `conjugated`

Currently, `conjugated` has two cases.

1. If the input has accessor type `conjugated_accessor<NestedAccessor>`, then the result has accessor type `NestedAccessor`;

2. otherwise, if the input has accessor type `A`, then the result has accessor type `conjugated_accessor<A>`.

This is correct behavior for any valid `value_type`, because `conjugated_accessor::access` uses *`conj-if-needed`* to conjugate each element. The exposition-only helper function object *`conj-if-needed`* uses namespace-unqualified `conj` if it can find it via argument-dependent lookup; otherwise, it is just the identity function. As P1673 explains, *`conj-if-needed`* exists for two reasons.

1. It preserves the type of its input (unlike `std::conj`, which returns `complex<T>` if the input is a floating-point type and therefore noncomplex).

2. It lets the library recognize user-defined types as complex numbers, as long as `conj` can be found for them via argument-dependent lookup.

The as-if rule would let `conjugated_accessor::access` skip calling *`conj-if-needed`* and just dispatch to its nested accessor if *`conj-if-needed`* would have been the identity anyway. However, the accessor type of the `mdspan` returned from `conjugated` is observable, so implementations cannot avoid using `conjugated_accessor`.

§ 4.3 Why change the current behavior?

The current behavior of `conjugated` is correct. The issue is that `conjugated` throws away the knowledge that its input `mdspan` views noncomplex elements. P1673 functions can optimize internally by using `conjugated_accessor::nested_accessor` to create a new `mdspan` for noncomplex `element_type`. However, that costs build time, increases the testing burden, and adds tedious boilerplate to every P1673 function.

This issue also increases the complexity of users' code. For example, users may reasonably assume that if they are working with noncomplex numbers and matrices that live in memory, then they only need to specialize their functions to use `default_accessor<ElementType>`. Such users will find out via build errors that `conjugated(x)` uses `conjugated_accessor` instead. Users may have to pay increased build times and possible loss of code optimizations for this complexity, especially if they write their own computations that use the result of `conjugated` directly as an `mdspan`.

As discussed in P1673 (see the section titled "Why users want to 'conjugate' matrices of real numbers"), linear algebra users commonly write algorithms that work for either real or complex numbers. The BLAS assumes this: e.g., `DGEMM` (Double-precision General Matrix-matrix Multiply) treats `TRANSA='C'` or `TRANSB='C'` (`'Conjugate Transpose'` in full) as indicating the transpose (same as `'T'` or `'Transpose'`). The Matlab software package uses a trailing single quote, the normal syntax for transpose in Matlab's language, to indicate the conjugate transpose if its argument is complex, and the transpose if its argument is real. Thus, we expect users to write algorithms that use `conjugate_transposed(x)` or `conjugated(transposed(x))`, even if those users never use complex number types or custom accessors. The current behavior means that such users will need to make their functions' overload sets generic on accessor type. This proposal would let those users ignore `conjugated_accessor` if they never use complex numbers.

§ 4.4 P1673 layouts and accessors are not “just tags”

Even though we propose to change the behavior of `conjugated`, `conjugate_accessor` needs to retain its current behavior. A key design principle of P1673 is that

> … each `mdspan` parameter of a function behaves as itself and is not otherwise “modified” by other parameters.

P1673’s nonwording section “BLAS applies `UPLO` to original matrix; we apply `Triangle` to transformed matrix” gives an example of the application of this principle.

Another way to say that is that the layouts and accessors added by P1673 are not “tags.” That is, P1673’s algorithms like `matrix_product` ascribe no special meaning to `layout_transpose`, `conjugated_accessor`, or `scaled_accessor`, other than their normal meaning as a valid `mdspan` layout or accessors. P1673 authors definitely intended for implementations to optimize for the new layouts and accessors in P1673, but a correct implementation of P1673 can just treat the `mdspan` types generically.

§ 4.5 Change: `conjugated(x)` may no longer have const `element_type`

Both `conjugated_accessor` and `scaled_accessor` have const `element_type`, to make clear that they are read-only views. This also avoids confusion about what it means to write to the complex conjugate of an element, or to the scaled value of an element. This proposal would change `conjugated(x)` to return `x` for `x` with noncomplex `value_type` and with accessors other than `conjugated_accessor<A>` for some `A`. As a result, the result of `conjugated(x)` would no longer have const `element_type` if `x` did not have const `element_type`.

We consider this change acceptable for two reasons.

1. *`in-vector`*, *`in-matrix`*, and *`in-object`* already do not need to have const `element_type`. Users can pass in views-of-nonconst `mdspan` as read-only vector or matrix parameters. Thus, making the `element_type` of `conjugated(x)` nonconst would not break existing calls to `linalg` functions that take input vector or matrix parameters.

2. `conjugated(conjugated(z))` for `z` with nonconst complex `element_type` already has nonconst `element_type`. Thus, generic code that depends on the `element_type` of the result of `conjugated` already cannot assume that it is const.

Regarding Reason (2), the current behavior of `conjugated` for an input `mdspan` object `x` with nonconst complex `element_type` is that

— `conjugated(x)` has const `element_type`, but

— `conjugated(conjugated(x))` has nonconst `element_type`.

This proposal would not change that behavior. The following example illustrates.

```
        constexpr size_t num_rows = 10;
        constexpr size_t num_cols = 11;
        vector<complex<float>> x_storage(num_rows * num_cols);

        // mdspan with nonconst complex element_type
        mdspan<complex<float>,
          dextents<size_t, 2>, layout_right,
          default_accessor<complex<float>>> x{
            x_storage.data(), num_rows, num_cols
        };

        // conjugated(x) has const element_type,
        // because `conjugated_accessor` does.
        auto x_conj = conjugated(x);
        static_assert(is_same_v<
          decltype(x_conj),
          mdspan<
            const complex<float>, // element_type
            dextents<size_t, 2>, layout_right,
            conjugated_accessor<default_accessor<complex<float>>>
          >
        >);
        // x_conj retains the original nested accessor and data handle,
        // even though these are both nonconst.
        static_assert(is_same_v<
          remove_cvref_t<decltype(x_conj.accessor().nested_accessor())>,
          default_accessor<complex<float>>
        >);
        // The data handle being nonconst means that we'll be able to
        // create conjugated(x_conj), even though conjugated(x_conj)
        // has nonconst data handle.
        static_assert(is_same_v<
          decltype(x_conj.data_handle()),
          complex<float>*
        >);
        // You can't modify the elements through x_conj, though,
        // because the reference type is complex<float>,
        // not complex<float>&.
        static_assert(is_same_v<
          decltype(x_conj)::reference,
          complex<float>
        >);

        // x_conj_conj = conjugated(conjugated(x));
        auto x_conj_conj = conjugated(x_conj);
        // x_conj_conj has x's original nested accessor type.
        static_assert(is_same_v<
          remove_cvref_t<decltype(x_conj_conj.accessor())>,
          default_accessor<complex<float>>
        >);
        // That means its element_type is nonconst, ...
        static_assert(is_same_v<
          decltype(x_conj_conj)::element_type,
```

```
          complex<float>
        >);
        // ... its data_handle_type is pointer-to-nonconst, ...
        static_assert(is_same_v<
          decltype(x_conj_conj.data_handle()),
          complex<float>*
        >);
        // ... and its reference type is nonconst as well.
        static_assert(is_same_v<
          decltype(x_conj_conj.access(declval<complex<float>*>(),
size_t{})),
          complex<float>&
        >);
```

§ 4.6 What if the input `mdspan` has `conjugated_accessor` with noncomplex `element_type`?

What should `conjugated(x)` do if `x` has accessor type `conjugated_accessor`, but noncomplex `element_type`? The current behavior already covers this case: just strip off `conjugated_accessor` and restore its nested accessor. This proposal does not change that.

Before this proposal, `conjugated` could produce an `mdspan` with accessor type `conjugated_accessor` but noncomplex `element_type`. The only thing that this proposal changes is that it eliminates any way for `conjugated` to reach this case on its own. Users could only get an `mdspan` like that by constructing an `mdspan` explicitly with `conjugated_accessor`, like this.

```
        std::vector<float> x_storage(M * N);
        std::mdspan x{x_storage.data(),
          std::layout_right::mapping{M, N},
          std::linalg::conjugated_accessor{std::default_accessor{}}};
```

There's no reason for users to want to do this, but the resulting `mdspan` still behaves correctly.

§ 5 Acknowledgments

Thanks to Tim Song (`t.canens.cpp@gmail.com`, Jump Trading) for making this suggestion during LWG review of P1673. We have his permission to acknowledge him by name for an LWG review contribution.

§ 6 Wording

> Text in blockquotes is not proposed wording, but rather instructions for generating proposed wording.

Change [linalg.conj.conjugated] paragraphs 1 and 2 to read as follows.

1 Let `A` be

(1.1) — `remove_cvref_t<decltype(a.accessor().nested_accessor())>` if `Accessor` is a specialization of `conjugated_accessor`;

(1.2) — otherwise, `Accessor` if `remove_cvref_t<ElementType>` is an arithmetic type or if the expression `conj(E)` is not valid with overload resolution performed in a context that includes the declaration `template<class T> conj(const T&) = delete;`;

(1.3) — otherwise, `conjugated_accessor<Accessor>`.

2 *Returns:*

(2.1) — `mdspan<typename A::element_type, Extents, Layout, A>(a.data_handle(), a.mapping(), a.accessor().nested_accessor())` if `Accessor` is a specialization of `conjugated_accessor`; otherwise

(2.2) — `a` if `remove_cvref_t<ElementType>` is an arithmetic type or if the expression `conj(E)` is not valid with overload resolution performed in a context that includes the declaration `template<class T> conj(const T&) = delete;`; otherwise,

(2.3) — `mdspan<typename A::element_type, Extents, Layout, A>(a.data_handle(), a.mapping(), conjugated_accessor(a.accessor()))`.